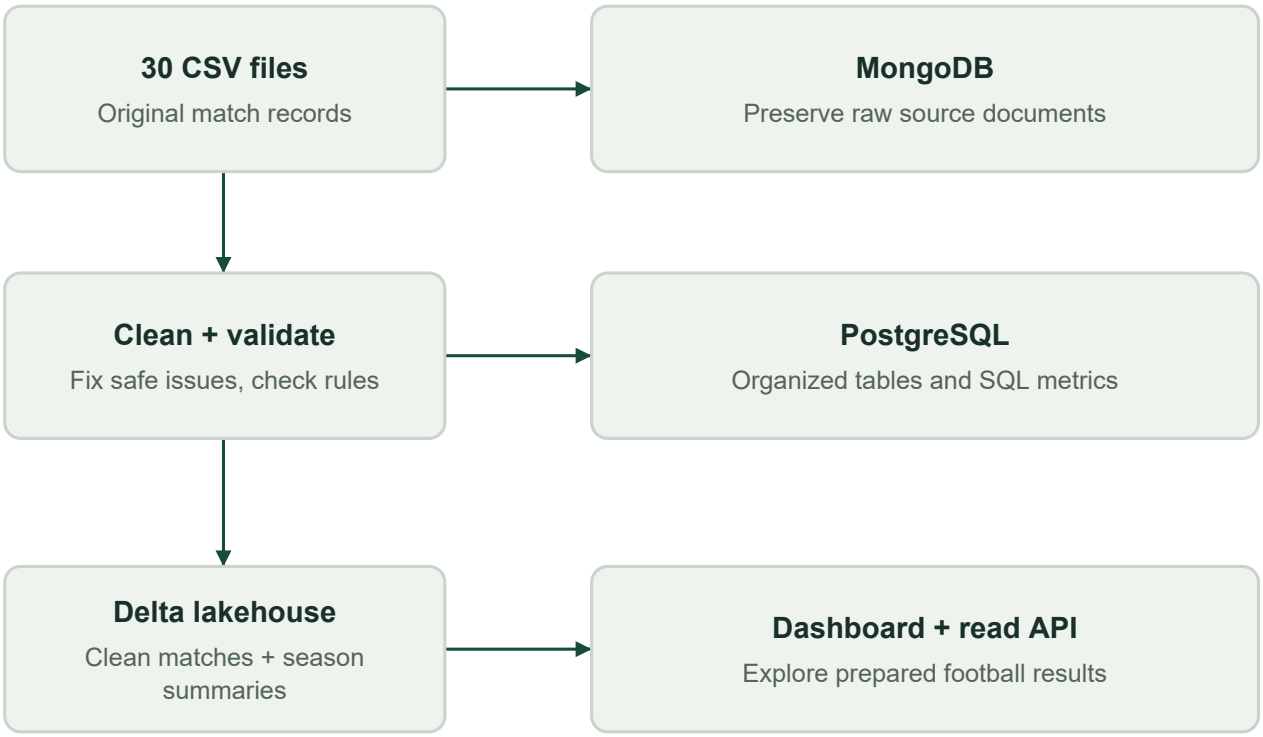
# How the project works

A visual guide to the actual football platform, with the technical terms explained

PitchPulse360 turns football match files into checked data and useful football charts. The supplied history covers five leagues over six seasons: **10,781 matches and 30,186 goals**. These counts come from the saved verified run.




**Read the arrows as movement of data.** The raw document branch preserves the source. Cleaned data feeds PostgreSQL and the lakehouse. The football dashboard and original read API use prepared lakehouse data, with DuckDB helping query it.

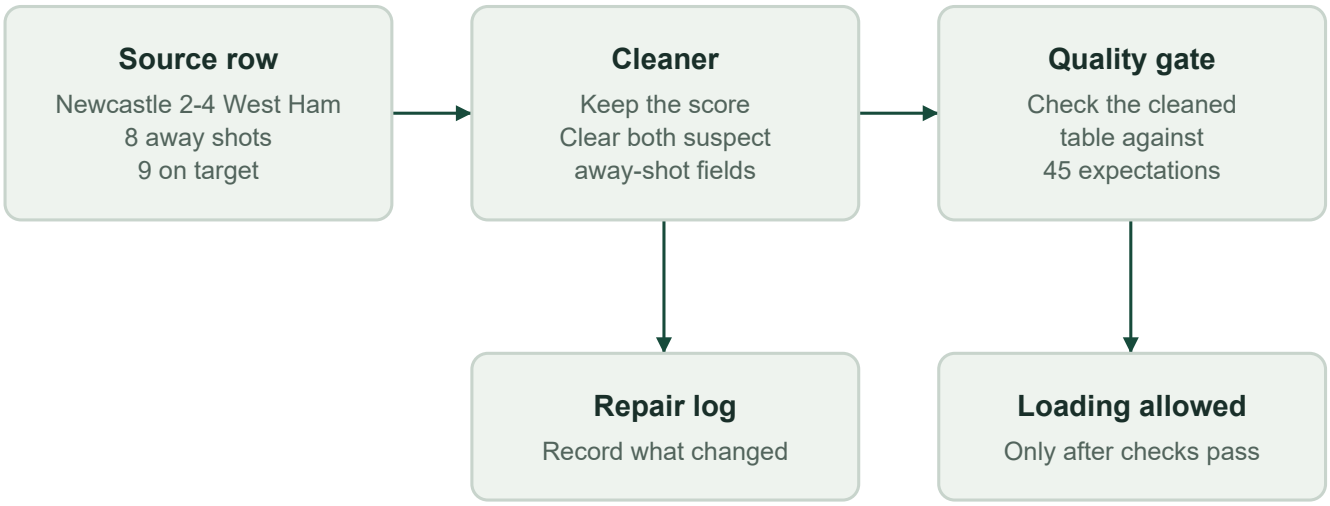
## Two routes, two jobs

**Batch route:** process the historical files together. **Event route:** accept an individual JSON event through FastAPI and RabbitMQ into PostgreSQL staging. The second route does not automatically rewrite historical standings.

Project evidence: platform_lab/flow.py; dashboard/app.py; serving/api.py; results/verification.json

# Reading, cleaning and checking

Experiments 1 and 13 / follow one actual source problem




The source row is from 15 August 2021. Nine shots on target cannot come from eight total shots. The program keeps the usable match result and marks the two inconsistent measurements as unknown. Unknown is different from zero.

## What the cleaner does

It trims extra spaces, standardizes selected team aliases, reads dates correctly, converts numeric values and identifies duplicate fixtures. Each row keeps its source filename and row number so a suspicious output can be traced back.

It also calculates total goals, goal difference and points. In this match, total goals are **2 + 4 = 6**. Newcastle gets 0 points and West Ham gets 3 points.

## What the quality gate proves

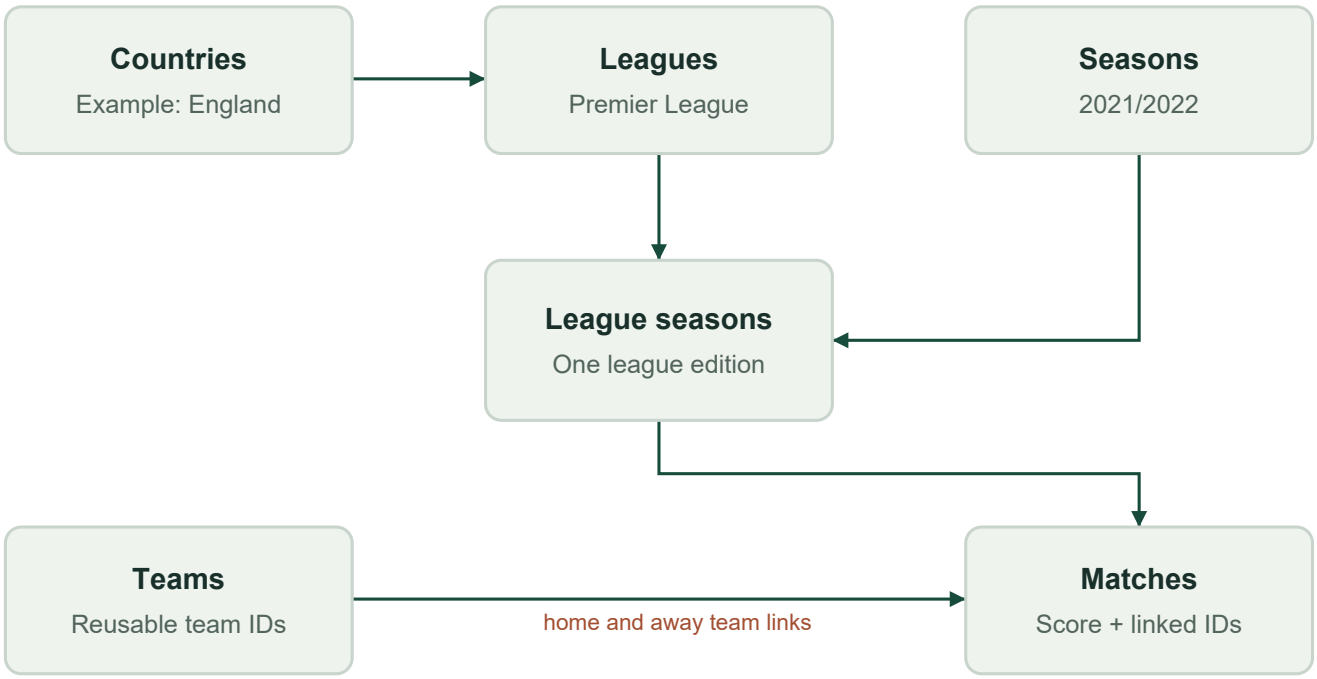
Required columns, valid types, allowed ranges, unique IDs and score/result agreement must pass. The clean data passes all **45 checks**. A separate damaged sample triggers **9 failed expectations**. A failed clean-data gate stops warehouse loading.

The saved cleaner repaired optional measurements in one row and rejected no complete matches. It retained 69 unusually high-scoring matches because unusual results can still be valid. This is a custom Pandas expectation suite, not the Great Expectations library.

Project evidence: platform_lab/cleaning.py; quality/football_expectations.json; results/eda/summary.json; results/repaired_measurements.csv

# Organized tables and safe loading

Experiments 2 and 3 / PostgreSQL is the relational warehouse




A team is stored once and reused by many matches. A match refers to two team IDs and a league-season ID. This reduces repeated descriptions and demonstrates a normalized core model.

| Term | Plain meaning |
|---|---|
| Primary key | The unique identity of one row |
| Foreign key | A reference to an existing row in another table |
| Constraint | A rule the database refuses to break |

## The load is a controlled replacement

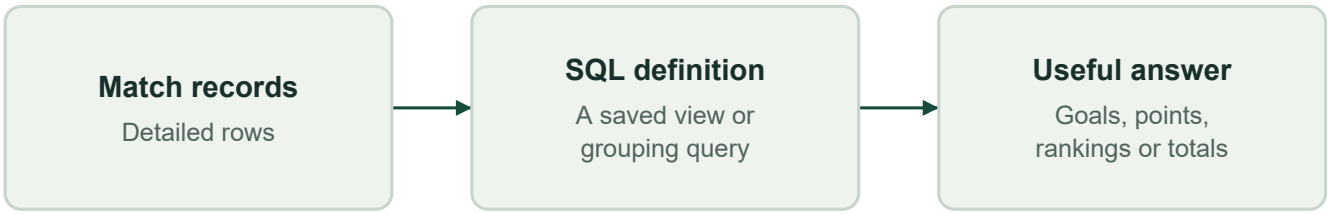
**COPY** transfers cleaned rows into a temporary table. The loader creates missing reference entries, joins their IDs and refreshes the historical snapshot. A **transaction** commits the database changes together or rolls them back on failure. Stable fixture IDs and unique constraints prevent duplicate historical matches on reruns.

The analytical fact table has six season partitions. Date/team indexes support selected lookups. A saved EXPLAIN plan shows how PostgreSQL actually runs a query. The transaction covers PostgreSQL work, not every other storage system.

Project evidence: models/postgres_schema.sql; platform_lab/postgres.py; results/postgres/summary.json

# Saved questions and summary levels

Experiments 4 and 5 / SQL views and OLAP




A **view** is a saved question. Everyone can reuse the same definition instead of rewriting the formula. A normal view does not precompute the answer and does not guarantee a faster query.

| Function | What it helps answer |
|---|---|
| DENSE_RANK | Where does a team rank by its season totals? |
| LAG / LEAD | What was the previous / next available season value? |
| Running SUM | How many points had the team accumulated by this match? |

## Three ways to ask for totals

| Operation | Levels returned in this project |
|---|---|
| ROLLUP | 30 league-season rows + 5 league totals + 1 overall total |
| CUBE | The same groups, plus 6 season totals |
| GROUPING SETS | Exactly the requested combinations of these groups |

PostgreSQL and DuckDB must return equal results. **OLAP** simply means looking at the same facts from different summary angles, such as league and season.

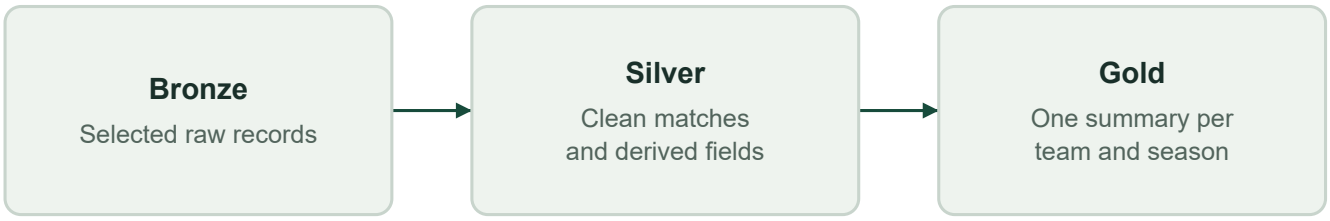
## Where the sales numbers come from

The project generates **6,000 synthetic ticket orders** for daily revenue, monthly growth and repeat-customer calculations. Paid quantity times price contributes revenue; refunded orders do not. Zero-sales months stay in the calendar. These are teaching orders, not actual club finances.

Project evidence: models/metric_views.sql; platform_lab/olap.py; results/metrics/summary.json; results/olap/comparison.json

# Files with useful layers and history

Experiment 8 / the Delta lakehouse




**Parquet** stores values by column. **Delta Lake** adds a transaction log to its data files, so a table can retain a history of committed changes. The local deltalake library handles this without a Spark cluster.

## How Gold calculates a league table

For each team, combine its home and away games. Count wins, draws and losses. Add goals scored and conceded. Calculate **points = 3 x wins + draws**. Sort by points, goal difference and goals scored. Official deductions and every competition exception are not fully modeled.

| Demo version | What changes | Rows after change |
|---|---|---|
| 0 | Create demo-1 and demo-2 | 2 |
| 1 | Add demo-3 | 3 |
| 2 | Change demo-1 from 2 goals to 3 | 3 |
| 3 | Delete demo-2 | 2 |

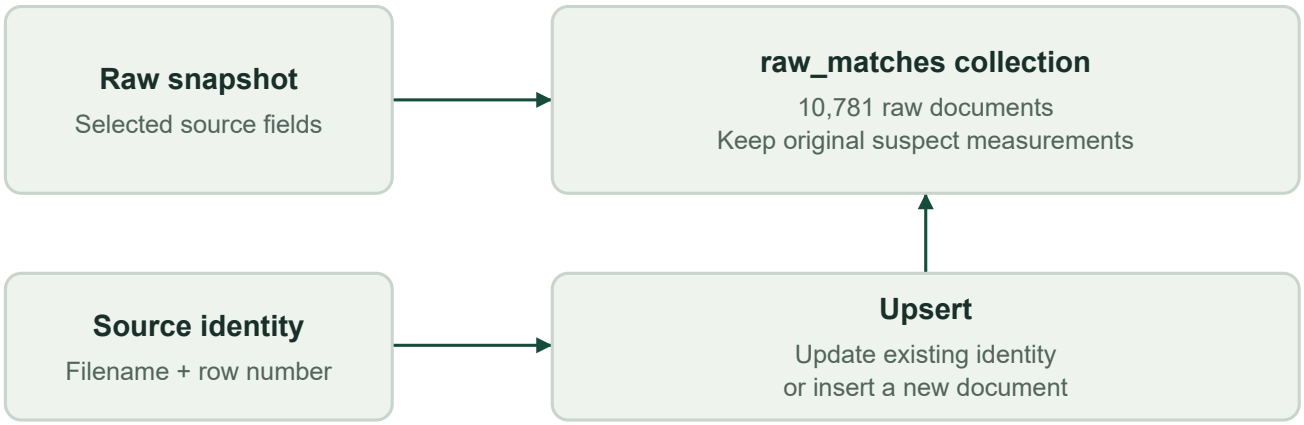
Reading version 0 still returns its original two rows. This is **time travel**: read an earlier table state. Two incompatible writes fail and leave the version unchanged. The experiment uses a separate tiny table, so it does not damage historical matches.

These checks cover the incompatible writes attempted in the demo. The full snapshot builder deliberately allows schema replacement, so the project does not claim every possible write mode blocks all schema changes.

Project evidence: storage/lakehouse_manager.py; platform_lab/delta_experiment.py; results/delta_lab/summary.json

# Keeping the original documents

Experiment 7 / MongoDB and CRUD




MongoDB stores document-shaped records with nested fields. PostgreSQL is useful for strict relationships; MongoDB provides a flexible original-document copy. Each has a different purpose.

The raw document can still contain the original eight-shots/nine-on-target inconsistency. The repaired analytical copy has unknown values. Keeping both lets you explain exactly what the cleaner changed.

## CRUD uses a separate scout-report example

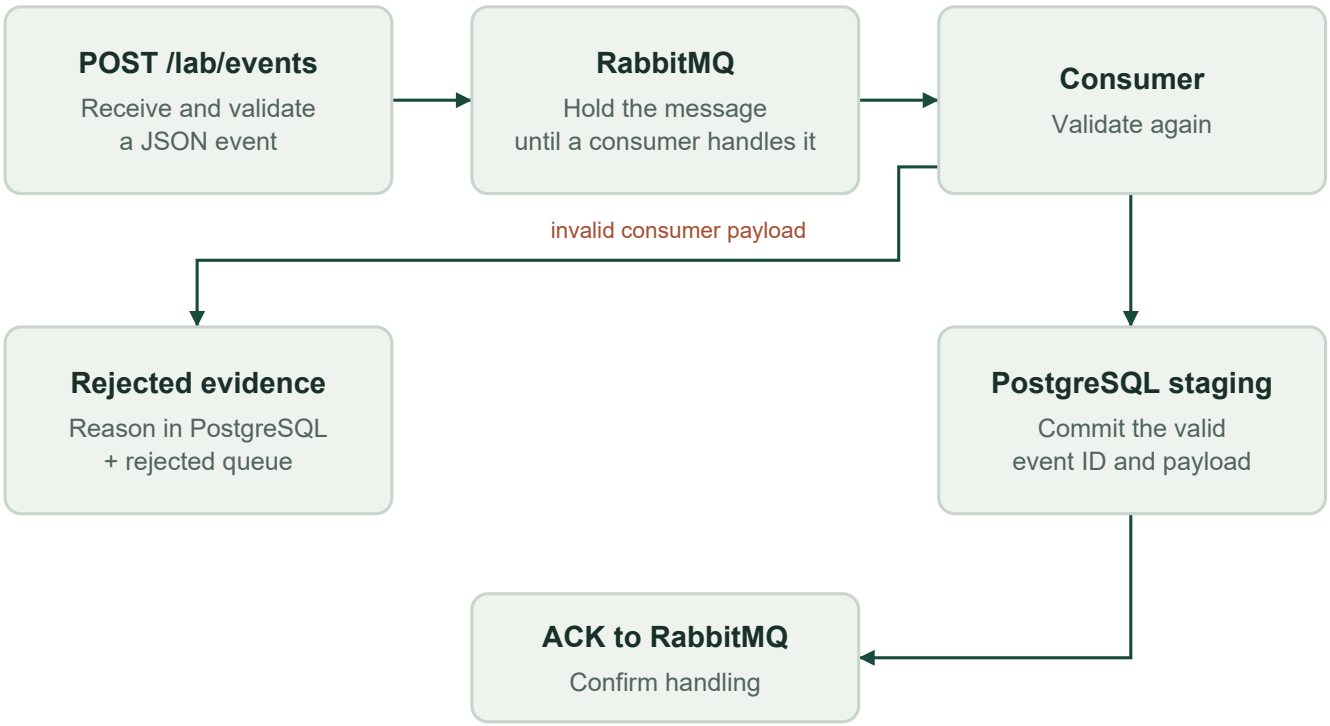
| Operation | What the demonstration does |
|---|---|
| Create | Insert a report with nested notes and tags |
| Read | Filter a report and return selected fields |
| Update | Change rating, nested review details and tags |
| Delete | Remove the demonstration report and confirm absence |

A **projection** means selecting which fields to return. An **index** helps a database find matching records. Compound indexes cover combinations such as league plus season. A saved query plan helps check whether MongoDB actually used an index.

Project evidence: platform_lab/mongo.py; results/mongo/crud_and_index_evidence.json; results/last_run.json

# An incoming event and its safety checks

Experiments 9 and 10 / FastAPI, RabbitMQ and staging




**JSON** is a structured message. Required fields include event ID, teams, date, goals and result. FastAPI rejects invalid requests with 422. It returns 202 only after the broker confirms publication. That means queued, not necessarily already stored.

| Situation | How the project responds |
|---|---|
| Same event ID arrives again | Unique event ID ignores another valid staging insert |
| Database fails | Do not acknowledge success; requeue for retry |
| Consumer receives bad data | Record the reason and send it to the rejected queue |

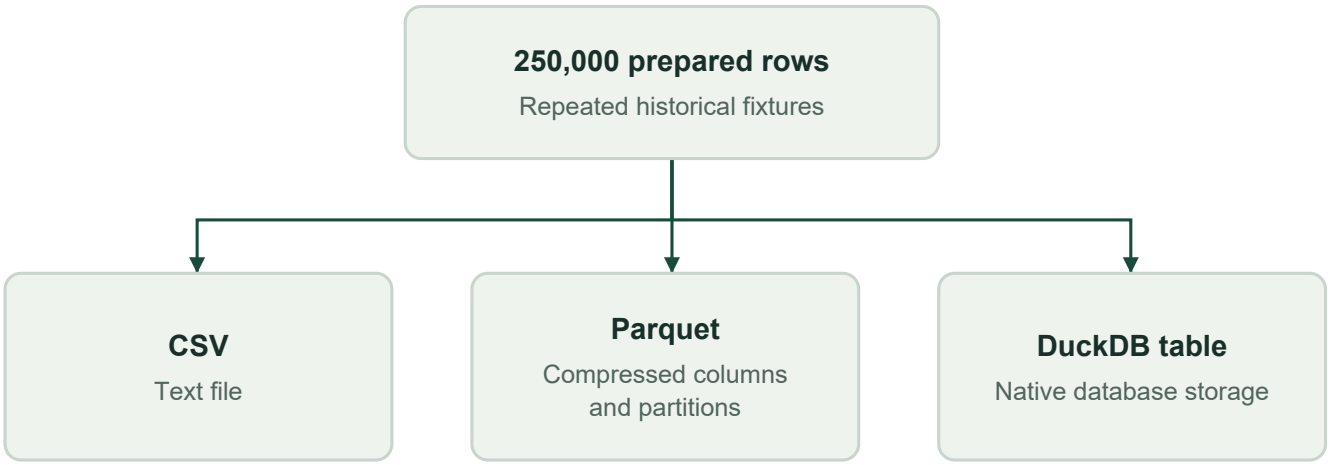
The consumer commits before sending **ACK**, meaning acknowledgement. A crash after commit can cause another delivery, but the event ID prevents a second valid row. This is **at-least-once delivery with an idempotent write**. Rejection audit records can repeat during a crash.

**Staging is a waiting area.** These replayed events do not automatically change Gold standings. A later reconciliation and promotion step would need to decide how events update match history.

Project evidence: platform_lab/api.py; platform_lab/streaming.py; results/last_run.json

# The same work with different storage

Experiment 11 / a local DuckDB performance demonstration




DuckDB queries each format with thread limits of 1 and 4. That gives **six modes**. Each receives one warm-up and five measured runs. The code first checks that every mode returns the same answer.

| Storage | 1 thread, median ms | 4 threads, median ms |
|---|---|---|
| CSV | 167.156 | 166.694 |
| Parquet | 152.962 | 120.000 |
| DuckDB | 9.515 | 9.037 |

These are the saved run's local, warm-cache observations. They include the configured storage path. They are not a promise that a product or four threads will always win.

## Why storage can affect a query

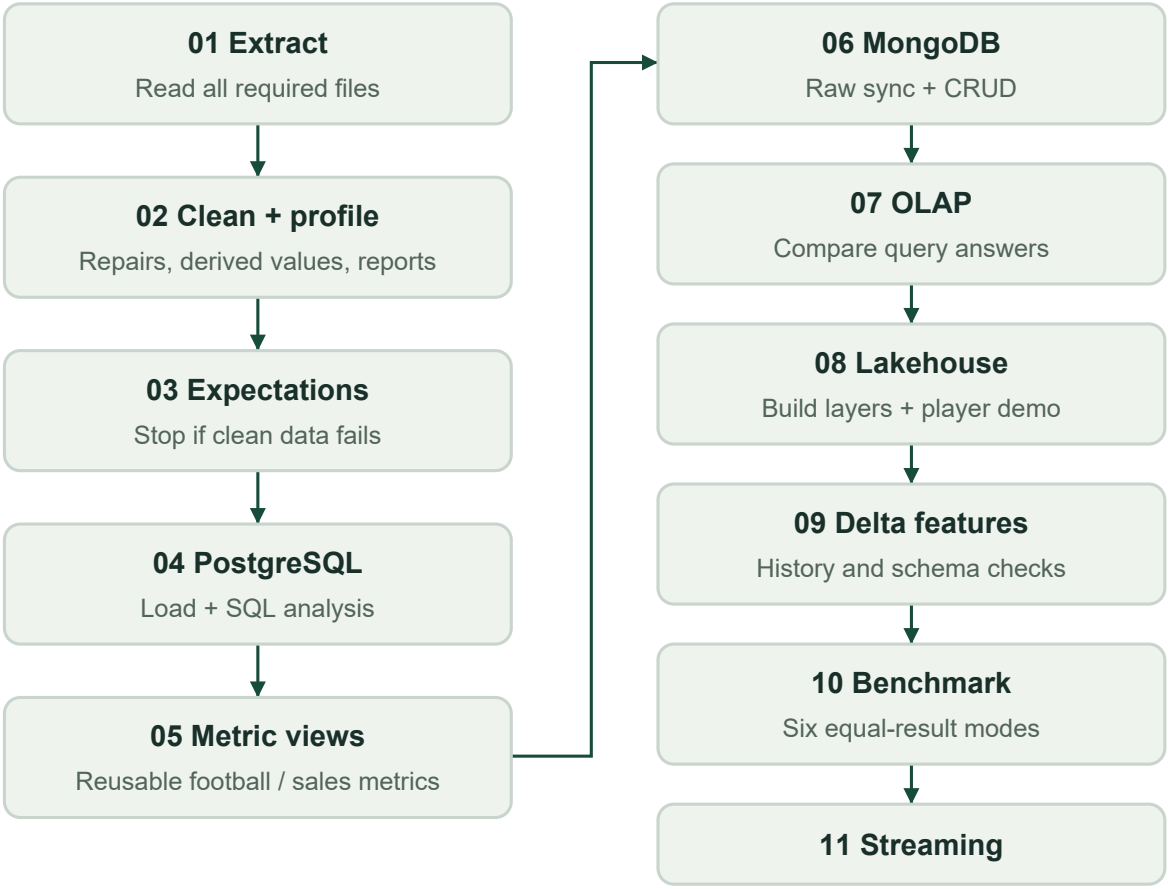
A columnar file can skip irrelevant columns. Partition pruning can skip unrelated groups of files. A native database has its own organized storage. Four threads allow work on several CPU workers on **one computer**; this is not a distributed Spark/Hadoop cluster.

The benchmark repeats existing fixtures to create a controlled workload. It does not add 250,000 real matches. Repeated rows compress particularly well, so its file-size ratio should not be treated as a prediction for new data.

Project evidence: platform_lab/benchmark.py; results/benchmark/summary.json

# Who runs the steps in order?

Experiments 12 and 14 / Prefect coordinates the batch




Prefect starts these tasks in sequence and records their status and logs. Several tasks retry temporary failures. A failed clean-data gate stops without retrying. A permanent failure eventually fails the flow.

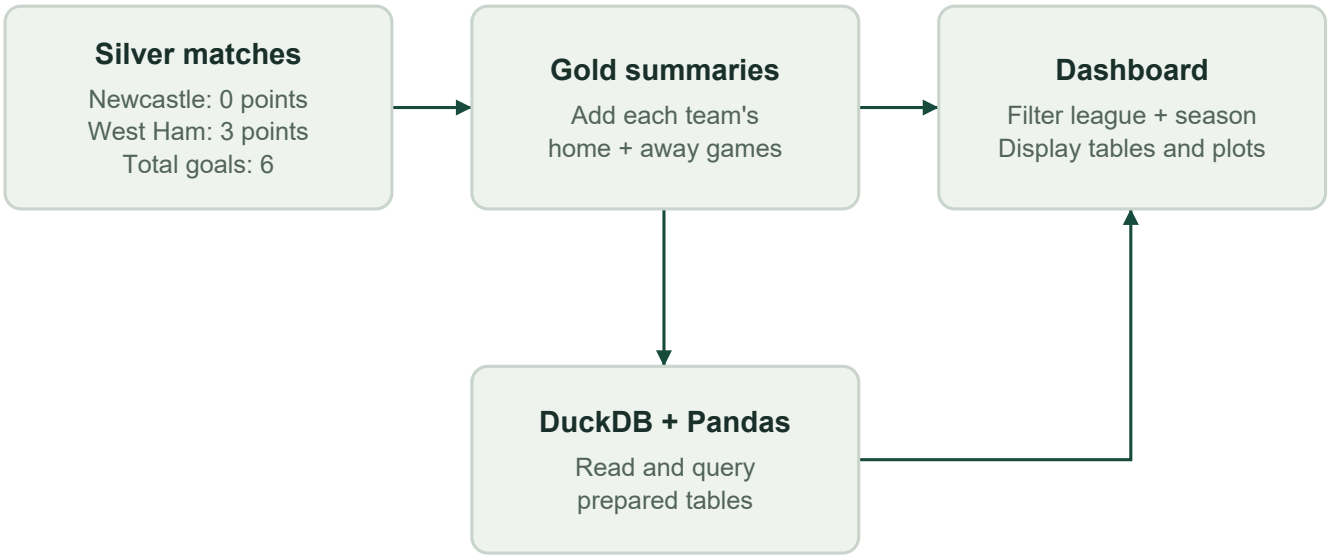
The default deployment runs every **3,600 seconds**. A run limit and file lock prevent scheduled and manual batches from replacing outputs together. PostgreSQL also locks its transaction. Scheduled work needs Docker and the services to stay running.

PostgreSQL, MongoDB and Delta do not share one global transaction. A later task failure can leave an earlier storage output updated. A future improvement is a shared published batch version across outputs.

Project evidence: platform_lab/flow.py; platform_lab/verification.py; results/prefect/scheduled_run_evidence.json

# How a match becomes a chart

Experiment 6 / the dashboard reads prepared data




The football page reads Silver and Gold Delta data. DuckDB queries registered dataframes. The read API also uses Gold. The dashboard does not query PostgreSQL for every chart. Its data-loading cache has a 60-second TTL, so a refresh after cache expiry may be needed to see a rebuilt snapshot.

| Page feature | What it answers |
|---|---|
| Standings + goal matrix | Who has the points? Who scores or concedes more? |
| Head-to-head + records | What happened in the included meetings and seasons? |
| Player heatmap + radar | How can spatial and profile visualizations work? |
| Engineering Lab | What evidence supports each engineering experiment? |

The saved run has **34 manual player profiles and 5,741 generated spatial actions**. KDE spreads action points into a smooth density heatmap. Its bright areas describe generated activity, not measured tracking. Radar scores are project-defined indices.

Standings use simplified sorting and qualification/relegation labels. Official deductions and league-specific exceptions are not fully modeled. Records refer to the six included seasons, not all football history.

Project evidence: dashboard/app.py; dashboard/pages/Engineering_Lab.py; storage/lakehouse_manager.py; results/last_run.json

# All 15 experiments in one place

A checklist you can use while preparing for questions

| No. | Experiment | Where it appears in this guide |
|---|---|---|
| 1 | Explore and clean data | Page 2 |
| 2 | Relational model and SQL | Pages 3-4 |
| 3 | CSV to PostgreSQL ETL | Page 3 |
| 4 | Reusable SQL metric views | Page 4 |
| 5 | Multidimensional totals | Page 4 |
| 6 | Analytics dashboard | Page 10 |
| 7 | MongoDB CRUD and indexes | Page 6 |
| 8 | Delta versions and old reads | Page 5 |
| 9 | REST API data capture | Page 7 |
| 10 | Queued event ingestion | Page 7 |
| 11 | Parallel local processing | Page 8 |
| 12 | Workflow scheduling | Page 9 |
| 13 | Data expectation suite | Page 2 |
| 14 | Connected batch pipeline | Pages 1 and 9 |
| 15 | Combined data platform | Pages 1 and 12 |

## Terms in ordinary words

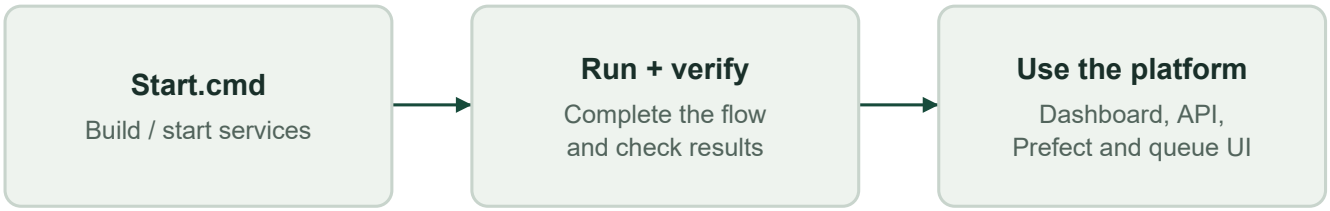
**ETL:** read, prepare and load data. **Schema:** the structure and rules of a table. **View:** a saved query. **Partition:** a group of stored rows or files. **Provenance:** where a record came from. **Consumer:** the worker that reads queue messages.

The project implements the listed teaching capabilities. Its scope limits still matter: generated player positions, synthetic orders, replayed events, local processing and simplified football rules.

Project evidence: ALL_15_EXPERIMENTS.md; results/verification.json; results/completion_summary.json

# Running it and explaining the result

Experiment 15 / the complete local product




| Command or address | Purpose |
|---|---|
| Start.ps1 -Action Start | Build and initialize the local product |
| Start.ps1 -Action Run | Repeat the complete batch and verification |
| Start.ps1 -Action Test | Execute the tests |
| Start.ps1 -Action Stop | Stop services while keeping database volumes |
| 127.0.0.1:8502 | Football dashboard and Engineering Lab |
| 127.0.0.1:8002/docs | API documentation and event submission |
| 127.0.0.1:4201 | Prefect run history |
| 127.0.0.1:15673 | RabbitMQ management |

Run commands from the extracted project folder in PowerShell with the **.** prefix, for example **.\Start.ps1 -Action Run**. Initial Docker image and dependency downloads need internet. These local addresses work while the relevant services are running.

## What the saved evidence establishes

The stored verification records 10,781 PostgreSQL matches, 10,781 MongoDB documents, matching analytical answers and eleven completed Prefect tasks. The recorded test suite has 30 passing tests. This guide explains that saved evidence; generating these PDFs does not rerun the pipeline.

## The next useful improvements

Add official competition rules, observed player events and a controlled event-to-history promotion step. Then add incremental updates, consistent published batch versions, real authentication, managed secrets, monitoring and backups for deployment.

Project evidence: Start.ps1; compose.yaml; results/verification.json; results/completion_summary.json; WORKFLOW_GUIDE.md